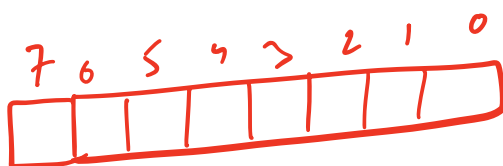


# String

- ↳ Array of chars
- ↳ Sequence of chars

chars ?  $\rightarrow$  1B / 2B

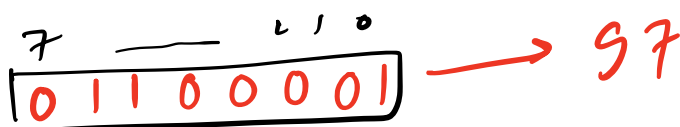
char x = 'a';



## ASCII System

|          |           |          |
|----------|-----------|----------|
| 'A' → 65 | 'a' → 97  | '0' → 48 |
| 'B' → 66 | 'b' → 98  | '1' → 49 |
| ...      | ...       | ...      |
| 'Z' → 90 | 'z' → 122 | '9' → 57 |

char x = 'a';



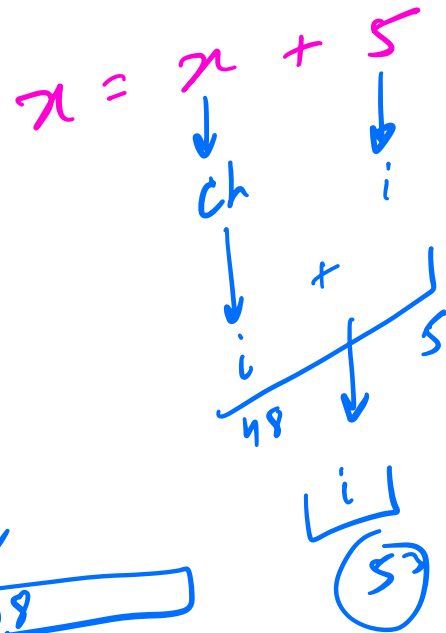
char x = '0';

print(x);

print(int(x))

x += 5;

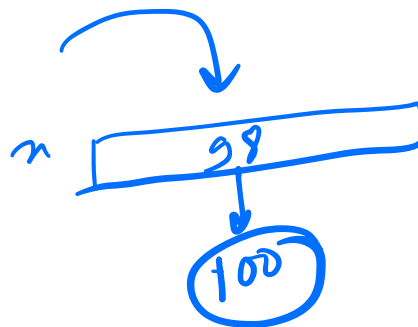
print(x);



char x = 'b';

x += 2;

print(x)



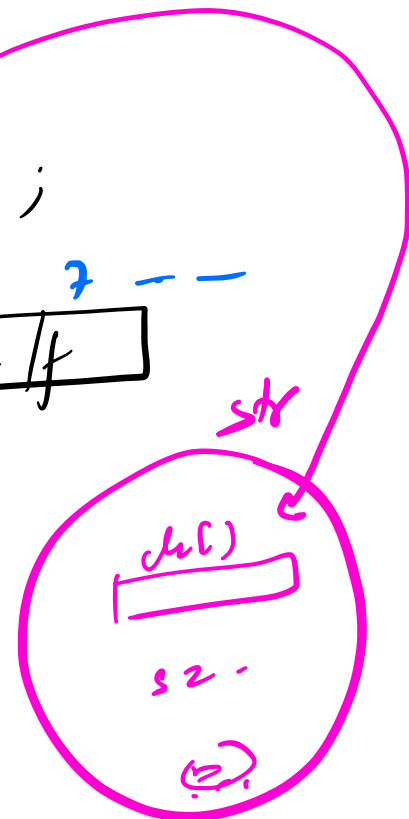
string s = "cdab0l@f";

char s[-] =

|   |   |   |   |   |   |   |   |     |
|---|---|---|---|---|---|---|---|-----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | ... |
| c | d | a | b | 0 | l | @ | f |     |

print(s[4])

[3] -



string s = "rahul";

string y = "yadav";

string z = s + y;

print(z)

"rahul" + "yadav"

"rahulyadav"

Q

Given a string.

↓  
alphabets

Toggle

every char

Cap ↔ lower case

I/P "aNaConDA"

↓  
O/P "AnAcONda"

[65 - 90]

Upper Case

A (65)

B (66)

C (67)

⋮

Z (90)

+32  
-32

+32  
-32

+32  
-32

[97 - 122]

lower Case

(97) a

(98) b

(99) c

⋮

(122) Z

// s → string

```
f(i=0; i < s.size(); i++) {  
    if (65 ≤ s[i] && s[i] ≤ 90) {  
        'A' s[i] += 32;  
    }  
    else {  
        'Z' s[i] -= 32;  
    }  
}  
return s;
```

→ N  
→ Upper Case  
32 → 'a' - 'A'  
K =  
TC = O(N)  
SC = O(1)

|          | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----------|---|---|---|---|---|---|---|---|
| 'A' : 65 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 'B' : 66 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| ⋮        | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| 'Z' : 90 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 |

|           | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-----------|---|---|---|---|---|---|---|---|
| 'a' : 97  | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| 'b' : 98  | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 |
| ⋮         | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| 'z' : 122 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |

toggle the 5<sup>th</sup> bit

```

for (i = 0 → s.size() - 1) {
    s[i] = (s[i] ^ 1 << 5);
}

```

return s;

**TC =  $O(N)$**   
**SC =  $O(1)$**

Q Given a string. Sort it alphabetically!  
 ↓  
lower case

S = "a x b a a c d"; I/P  
 SORT  
 a a a b c d x ✓ O/P

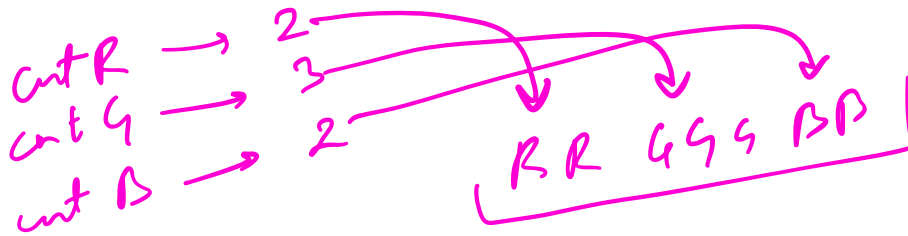
1) S.sort() + Custom comparator

**TC =  $O(N \log N)$**   
**SC =  $O(1)$**

2)

"R G B R G G B"

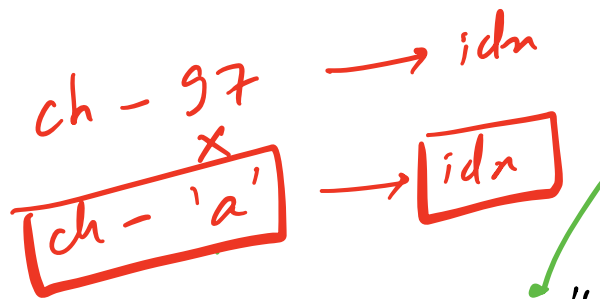
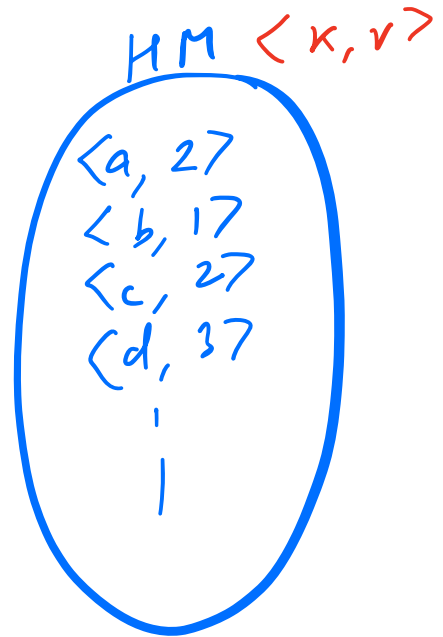
R/G/B



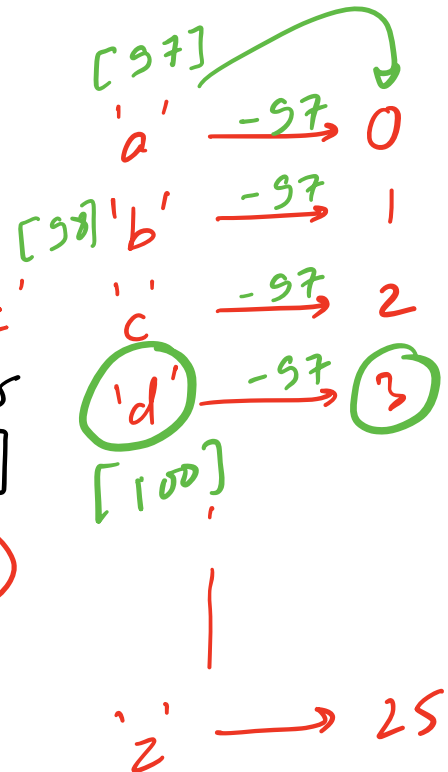
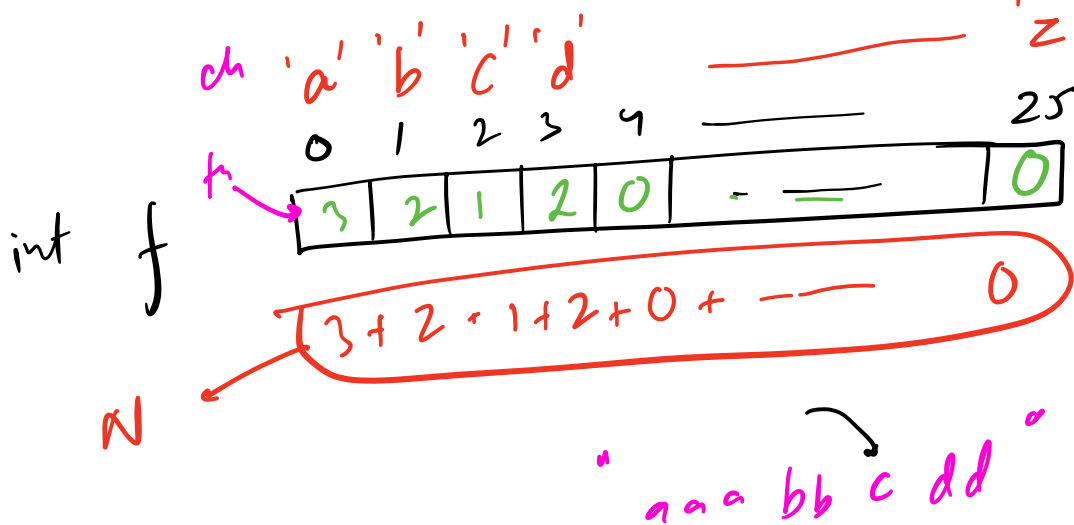
S = "b a d c d d a c"

'a' b c d e f  
 "a a b c c d d d"

→ 'z'



S = "a d d b a c b a"



```

int f[26] = {0};
for (i = 0; i < s.size(); i++) {
    idn = s[i] - 'a';
    f[idn]++;
}

```

$\rightarrow N$

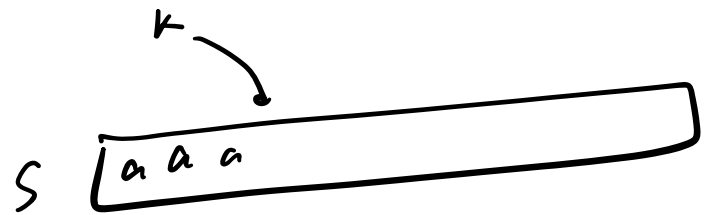
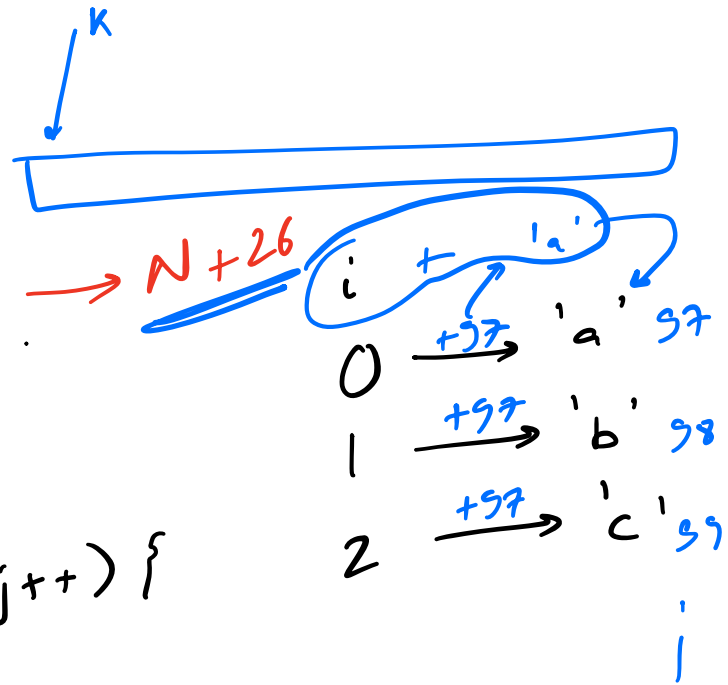
$\rightarrow f[s[i] - 'a']++;$

$k = 0;$

```

for (i = 0; i < 26; i++) {
    times = f[i];
    char ch = i + 'a';
    for (j = 0; j < times; j++) {
        s[k] = ch;
        k++;
    }
}

```



return s;

COUNT SORT

$SC = O(1)$

$O(k)$

$TC = N + N + 26$

$2N + 26$

$TC = O(N)$

$k$ : Alphabet Set Size

$O(N + k)$

## ① Substring

↳ A continuous part of a string

think of it like a subarray for a string

$S = "a \ b \ x \ \underline{c \ d} \ f"$

Q Given a string. check if any of its substring is a palindrome!

$S = "x \ a \ \textcircled{b} \ c"$

A single char is a substring  
A single char is ALWAYS a PALINDROME!

ANS → true

Q Given a string. check if it is a palindrome!  
↓  
substring [L, R]

"a b a b a"  


" 0 1 2 ... N-1 "  
 a x b x a  
 i j

```

i = 0, j = N-1;
while (i < j) {
  if (s[i] != s[j]) {
    return false;
  }
  i++;
  j--;
}
return true;

```

a b b a  
 ↕ ↕

$i > j$   
 STOP

a b c b a  
 ↑↑

$i = j$   
 STOP

$TC = O(N)$   
 $SC = O(1)$

0 1 2 3 4  
 S: a b c b d

L, R  
 true ← [1, 3]  
 false ← [2, 4]

$TC = O(R-L)$

Q Given a string. Find the largest palindromic len of substring!

$s = a b a c a b$

1  
2  
3  
4  
5 → ANS

I) BF

ANS = 0;

f(L = 0 → N-1) {

f(R = L → N-1) {

[L, R]

if (isPalindrome(S, L, R) == true) {

ANS = MAX(ANS, R - L + 1);

}

}

}

return ANS;

TC =  $O(N^3)$

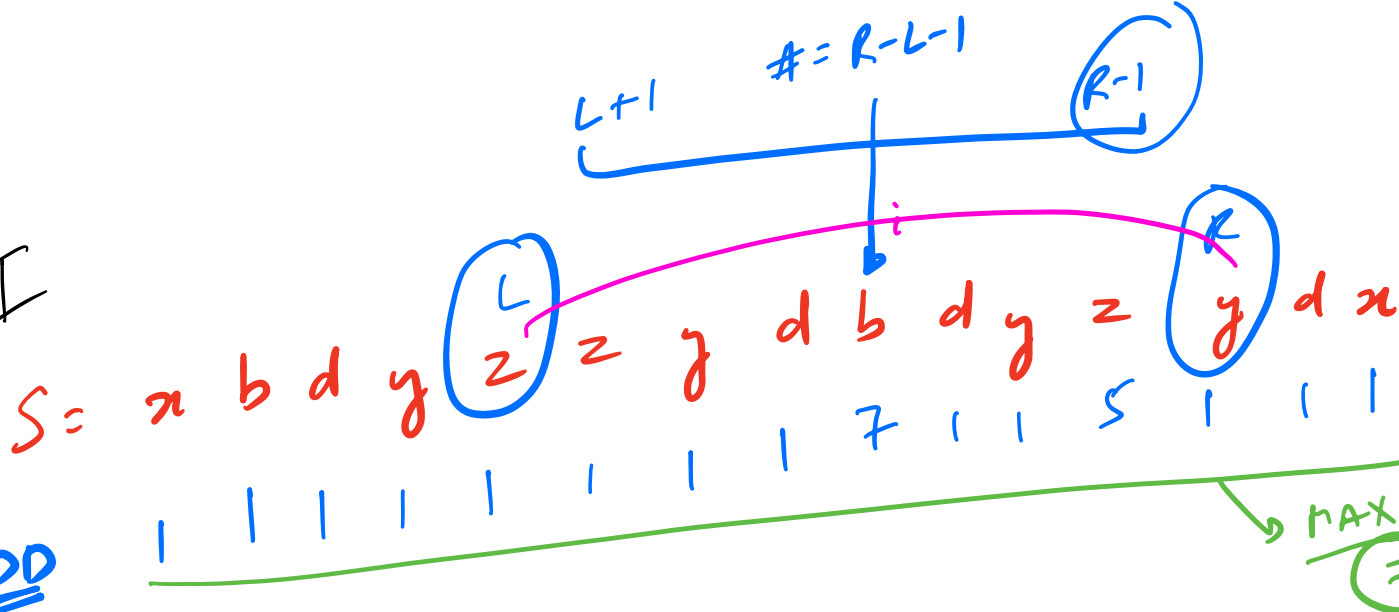
SC =  $O(1)$



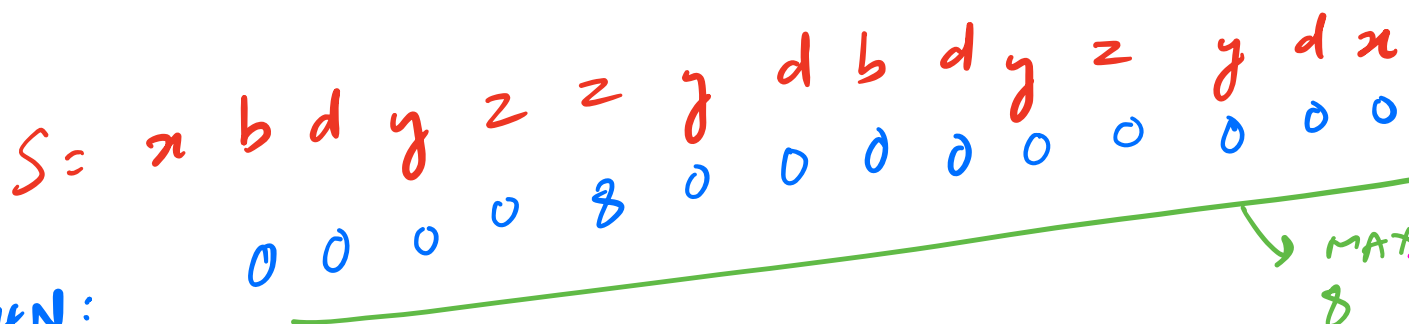
$\sim O(N)$

II

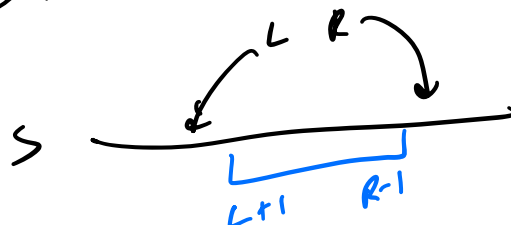
ODD



EVEN:



```
int expand (S, L, R) {
    while (L >= 0 && R < S.size()) {
        if (S[L] != S[R]) {
            break;
        }
        L--;
        R++;
    }
    return R - L - 1;
}
```



MAX  
8

ANS = 0;

f(i = 0  $\longrightarrow$  s.size() - 1) {

odd = expand(s, i-1, i+1);

even = expand(s, i, i+1);

ANS = MAX(ANS, odd, even);

}

return ANS;

**TC =  $O(N^2)$**

**SC =  $O(1)$**

| DP       | BS + HASH | MANACHER |
|----------|-----------|----------|
| $O(N^2)$ | $O(N^2)$  | $O(N)$   |